

Tutorial 10 - Spatial Models

SFR 605

The objective of this week's tutorial is to familiarize yourself with the tools and techniques to do exploratory analyses for spatial data sets. These analyses are useful to gain a better understanding of the data and how it is structured in order to help with the design of more advanced models and should not be done in place of more careful modeling.

##Case Study: Ozone Concentrations

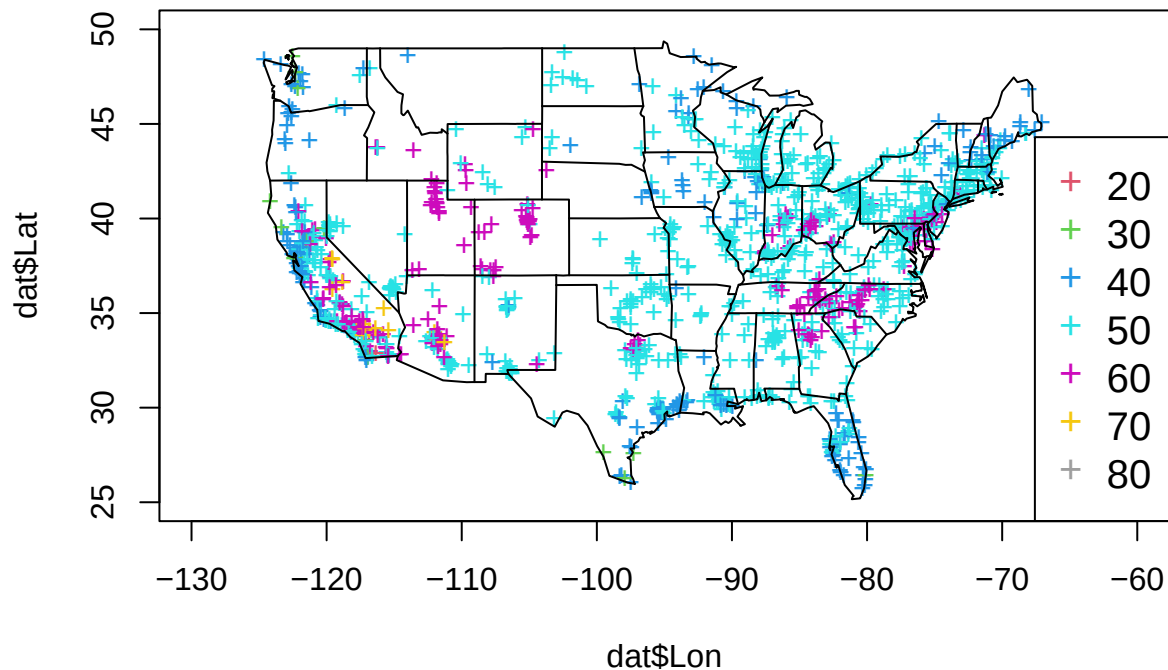
For this week's tutorial we will be looking at concentrations of ozone across the US based on the EPA's Air Quality System (AQS), which is a network of towers measuring ambient air quality (<http://www.epa.gov/ttn/airs/airsaqs/>). Data is collected on a number of pollutants and today we will focus on ozone (O3) because it is strongly related to not just human health but also ecosystem health—elevated ozone has significant impacts on crop productivity as well as the growth and survival of native flora and fauna. Specifically we will be looking at the annual means of hourly ozone concentration data for 2008. Units are in terms of parts per billion (ppb) and for reference the current health standard is 75 ppb. The file Ozone.RData contains a data frame `dat` that records the annual mean, standard deviation, and median ozone at 1225 stations along with each station's ID number and its latitude/longitude coordinates. The goal of today's analysis is to perform some basic exploratory analyses on the data—fit a trend surface, plot and fit the correlogram and variogram, and produce a kriged map of national ozone levels as well as the uncertainty associated with that map.

Lets start by loading the data and making a simple map of the observations. You will probably need to download one or both of the libraries indicated.

```
library(spatial)      ## spatial stats library
library(maps)          ## map of US
```

```
## Warning: package 'maps' was built under R version 4.5.2
```

```
load("data/Ozone.RData")      ## data
plot(dat$Lon,dat$Lat,xlim=c(-125,-65),ylim=c(25,50),asp=70/50,pch="+",col=dat$Mean/10)
map("state",add=TRUE)          ## add the states and then a legend
legend("bottomright",legend=2:8*10,col=2:8,pch="+",cex=1.25)
```



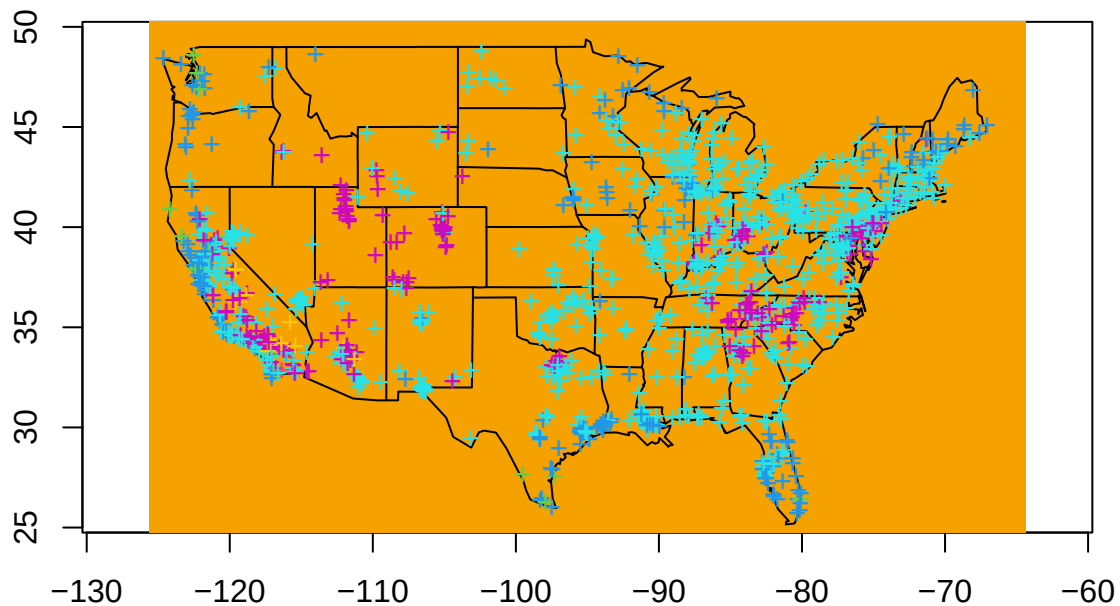
In the above most commands should be familiar or obvious. The one new bit worth mentioning is the `asp` variable set in `plot`. This locks the y/x aspect ratio so that the US doesn't get stretched/squished when you resize the window. 70/50 is the approximate ratio in km for 1 degree lat/lon within the study region (exact ratio varies with latitude).

Recall that spatial analysis, like time series analysis, assumes second order stationarity, so next we'll want to calculate a trend surface for the data to see to what extent we can detrend the data.

```
surf0 <- surf.ls(0,dat$Lon,dat$Lat,dat$Mean)  ## 0-degree polynomial surface
summary(surf0)                               ## ANOVA stats & AIC
```

```
## Analysis of Variance Table
## Model: surf.ls(np = 0, x = dat$Lon, y = dat$Lat, z = dat$Mean)
##           Sum Sq   Df Mean Sq F value Pr(>F)
## Regression    0.00    0      NaN     NaN   NA
## Deviation 57081.63 1224 46.63532
## Total       57081.63 1224
## Multiple R-Squared: 0,   Adjusted R-squared: 0
## AIC: (df = 1) 4707.888
## Fitted:
##      Min      1Q  Median      3Q      Max
## 49.88 49.88 49.88 49.88 49.88
## Residuals:
##      Min      1Q   Median      3Q      Max
## -29.8776  0.1224  0.1224  0.1224 20.1224
```

```
tr0 <- trmat(surf0,-125,-65,25,50,50)      ## project a 50x50 matrix within region
image(tr0,asp=70/50)                        ## make an color image map of the surface
map("state",add=TRUE)
points(dat$Lon,dat$Lat,pch="+",col=dat$Mean/10)
```



The **surf.ls** command fits a 2D polynomial trend to the data with the degree of the polynomial specified in the first term. The output of **surf.ls** provides two things that we care about, the parameters for the polynomial trend and the detrended residuals that we'll use to assess spatial autocorrelation.

The **trmat** (trend matrix) command takes the output of the **surf.ls** command and creates a list, here called **tr0**, which contains the x- and y-coordinates of the projection grid, and a matrix **z** that is the smoothed ozone value in each of the grid cells. Within this block of code remember that the 2nd through 5th terms in the **trmat** command define the boundary rectangle in terms of the latitude and longitude. The last argument, 50, gives the size of the matrix, meaning that the boundary rectangle of our map will be divided into a 50x50 grid.

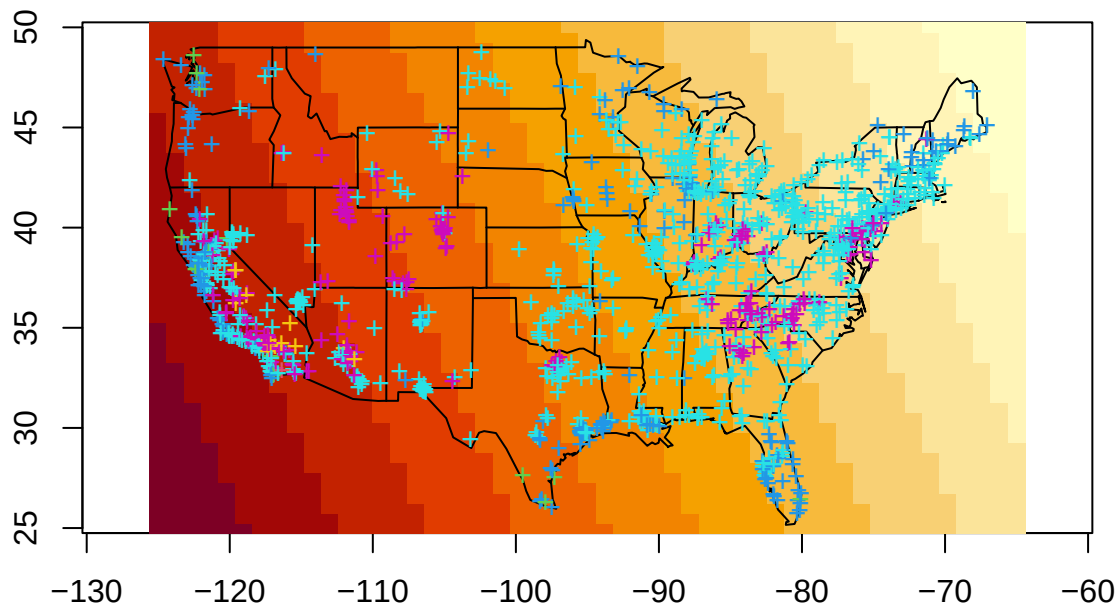
Task 1

1. Calculate the trend surface for higher order polynomials (at least 1st and 2nd) and include the plots in your answer. Do higher order polynomials capture large scale trends or are all the trends finer scale? What order polynomial would you choose to detrend your data?

```
surf1 <- surf.ls(1,dat$Lon,dat$Lat,dat$Mean)  ## 0-degree polynomial surface
summary(surf1)                               ## ANOVA stats & AIC
```

```
## Analysis of Variance Table
## Model: surf.ls(np = 1, x = dat$Lon, y = dat$Lat, z = dat$Mean)
##               Sum Sq   Df   Mean Sq   F value Pr(>F)
## Regression      3.405705    2   1.702852  0.03645673 0.9642
## Deviation  57078.226948 1222  46.708860
## Total      57081.632653 1224
## Multiple R-Squared:  5.966e-05,    Adjusted R-squared:  -0.001577
## AIC: (df = 3) 4711.815
## Fitted:
##      Min      1Q  Median      3Q      Max
##  49.78  49.84  49.86  49.93  50.10
## Residuals:
##      Min      1Q      Median      3Q      Max
## -29.95725   0.04832   0.14490   0.18660  20.06312
```

```
tr1 <- trmat(surf1,-125,-65,25,50,50)    ## project a 50x50 matrix within region
image(tr1,asp=70/50)                     ## make an color image map of the surface
map("state",add=TRUE)
points(dat$Lon,dat$Lat,pch="+",col=dat$Mean/10)
```

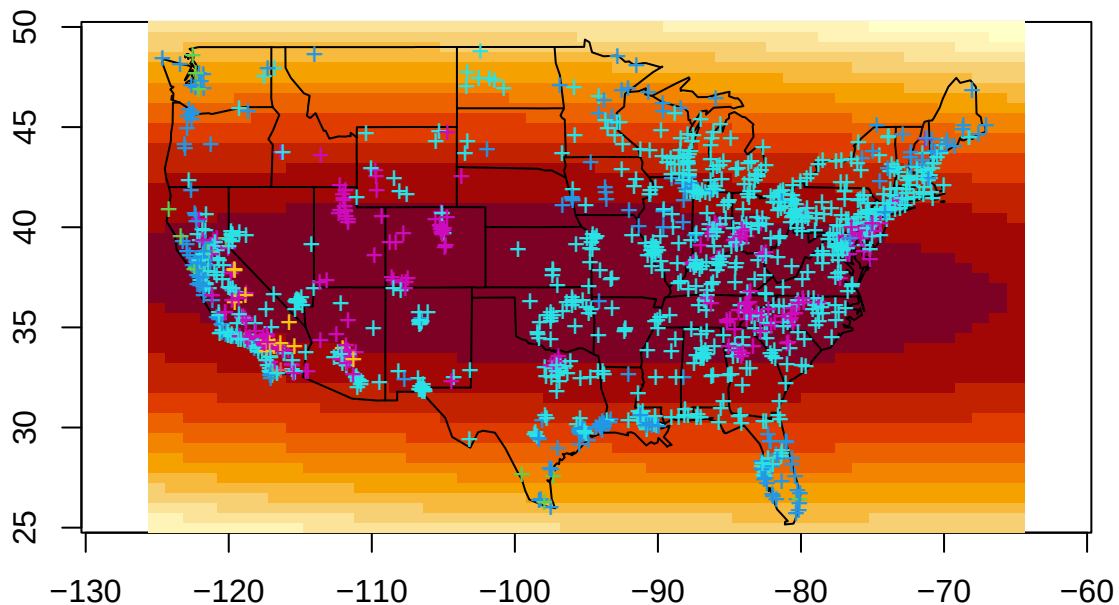


```
surf2 <- surf.ls(2,dat$Lon,dat$Lat,dat$Mean)    ## 0-degree polynomial surface
summary(surf2)                                   ## ANOVA stats & AIC
```

```
## Analysis of Variance Table
## Model: surf.ls(np = 2, x = dat$Lon, y = dat$Lat, z = dat$Mean)
```

```
##           Sum Sq   Df   Mean Sq F value    Pr(>F)
## Regression 10071.20    5 2014.23900 52.23005 < 2.22e-16
## Deviation  47010.44 1219   38.56476
## Total      57081.63 1224
## Multiple R-Squared: 0.1764, Adjusted R-squared: 0.1731
## AIC: (df = 6) 4480.1
## Fitted:
##      Min      1Q  Median      3Q      Max
## -2.611 49.231 50.597 51.400 52.210
## Residuals:
##      Min      1Q   Median      3Q      Max
## -30.3423 -1.8919 -0.5971  1.7639 32.6111
```

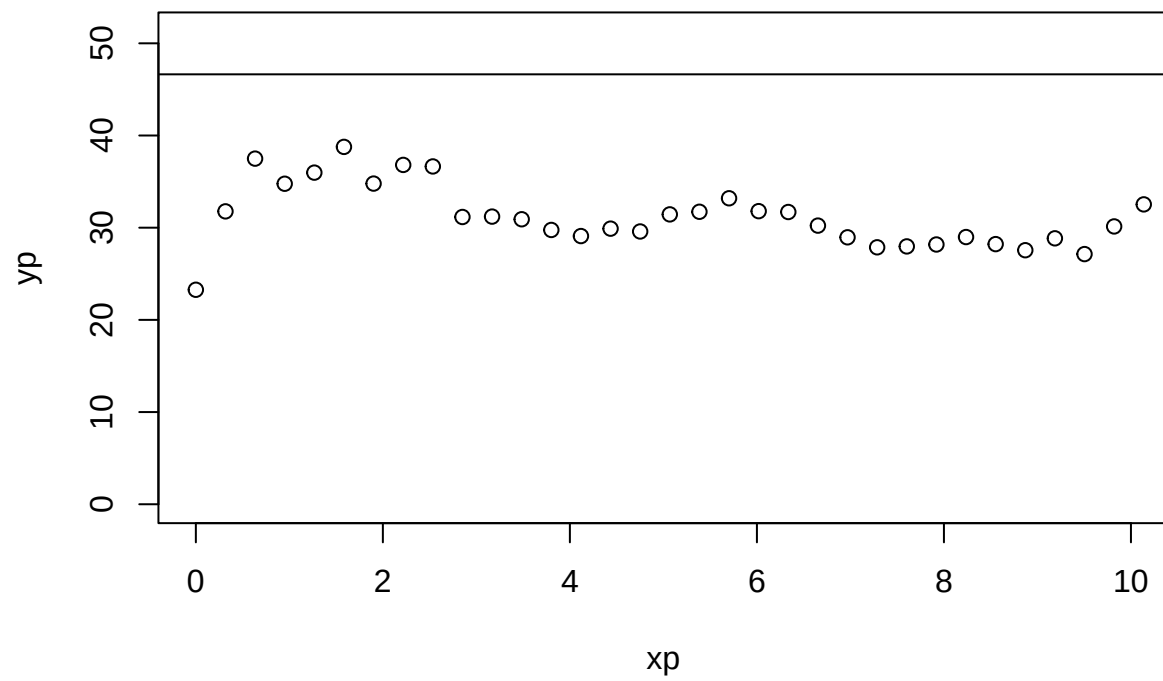
```
tr2 <- trmat(surf2,-125,-65,25,50,50) ## project a 50x50 matrix within region
image(tr2,asp=70/50)                  ## make an color image map of the surface
map("state",add=TRUE)
points(dat$Lon,dat$Lat,pch="+",col=dat$Mean/10)
```



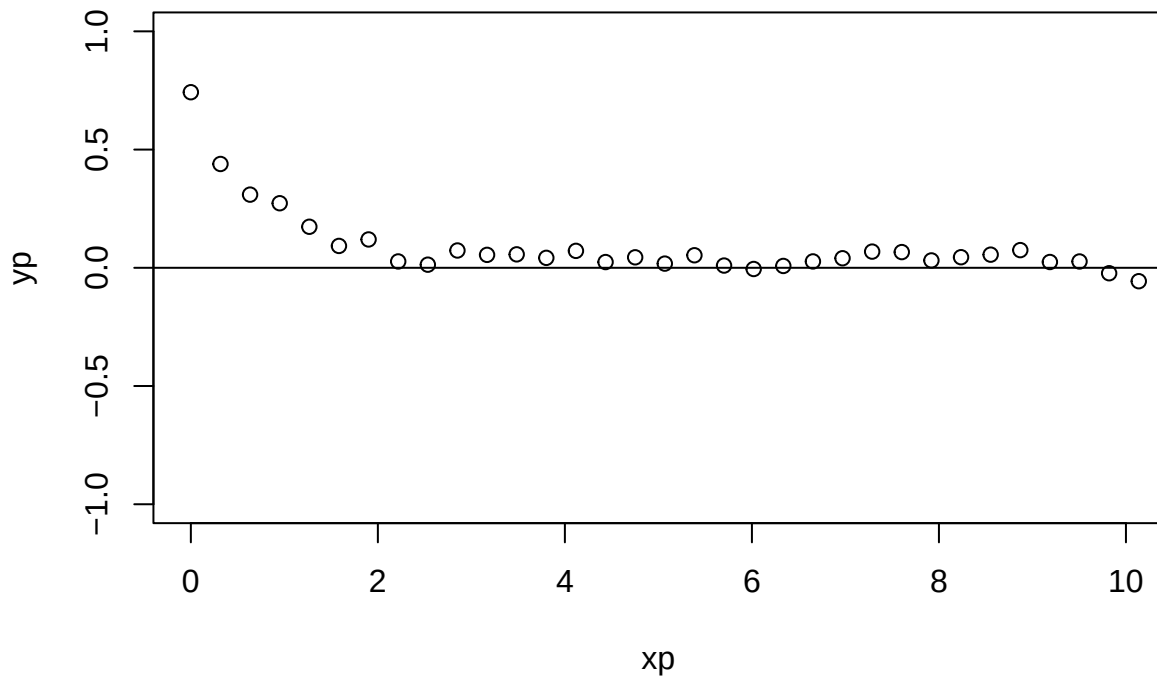
I am choosing a second order polynomial as finer resolution scale. This allows it to show a better pattern.

Now that we've calculated a trend surface let's calculate the correlogram and variogram in order to estimate the scale of spatial autocorrelation. *In the code below, make sure to substitute **surf0** with the polynomial surface you chose from Task 1:*

```
vg <- variogram(surf2,300,xlim=c(0,10),ylim=c(0,1.1*var(dat$Mean)))
abline(h=var(dat$Mean)) ## asymptotic variance
```



```
cg<- correlogram(surf2,300,xlim=c(0,10)) ## limit the range of radii to the initial decay
```



The variogram and correlogram both take the object output from `surf.ls` as their first argument, from which they are able to access the detrended residuals. The second argument in both functions is the number of bins that you want to include in calculating the variogram or correlogram. Be aware that a larger number of bins is not always better because the code will drop bins that have too few observations (<6). The third argument is just restricting our plots to the shorter distance lags where there is significant autocorrelation. The units in x are degrees so here “short” distance lags are still 100s of miles. The units in the y -axis for the variogram are in variance and the `abline` command provides the overall variance of the data which the variogram should asymptote to at large lags. The units on the y -axis of the correlogram are in terms of correlation (ranges from -1 to 1). Both the variogram and correlogram functions provide a plot by default. We are additionally assigning the output of these functions to a variable name so that we can access the data directly.

In order to interpolate the ozone values using kriging we need to fit a parametric model for the autocorrelation. Let’s derive a maximum likelihood estimator for the exponential correlation function `expcov`. `expcov` is a built-in R function and you can see the details of the computation by just typing `expcov` at the command line. In the following code we’ll fit the `expcov` function to the x and y values from the correlogram (`cgxandcgy`) assuming a Normal likelihood. Recall that x and y are not actually data but are the bin centers and correlation coefficients calculated from the binned data. Thus while the graphs may help suggest what functional form is best, we ultimately would want to fit the full spatial model directly to the data (which is beyond today’s tutorial).

```
rng <- 1:10                                ## define the distance bins we want to use for the fit
expfit <- function(theta){                  ## negative log likelihood function
  d = theta[1]
  alpha = theta[2]
  se = theta[3]
  sigma = theta[4]
```

```

    -sum(dnorm(cg$y[rng], expcov(cg$x[rng], d, alpha, se), sigma, log=TRUE))
  }
efit <- optim(c(2,0.1,1,0.01), expfit)    ## numerical optimization

## Warning in dnorm(cg$y[rng], expcov(cg$x[rng], d, alpha, se), sigma, log =
## TRUE): NaNs produced

efit

## $par
## [1] 0.98674918 0.17609328 0.86174070 0.02778215
##
## $value
## [1] -21.64453
##
## $counts
## function gradient
##      323      NA
##
## $convergence
## [1] 0
##
## $message
## NULL

```

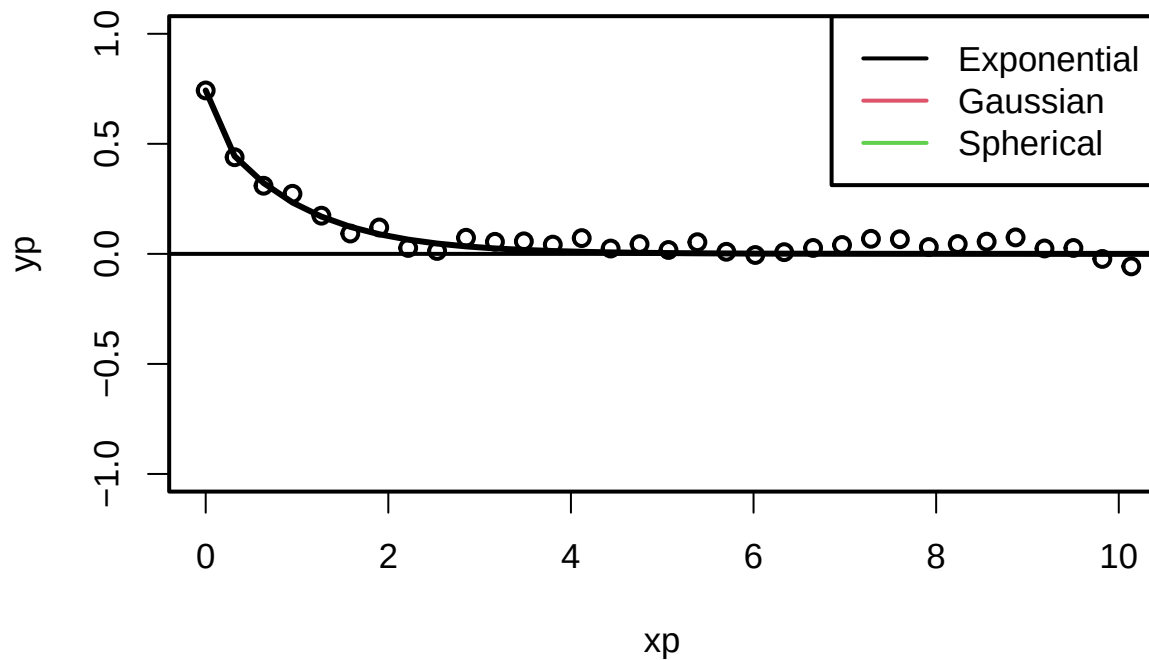
In the above example we restricted the range of the values from the correlogram to just the first 10 bins. This was done because we are really just interested in the initial decline phase and the number of bins would change depending on how long it takes the correlogram to converge to zero.

Next, let's plot the correlation functions over the correlogram to visually inspect the fits. You should extend this code to include any additional models you fit.

```

par(lwd=2, cex=1.1)
cg <- correlogram(surf2, 300, xlim=c(0,10))
lines(cg$x, expcov(cg$x, efit$par[1], efit$par[2], efit$par[3]), lwd=3)
#lines(cg$x, gaucov(cg$x, gfit$par[1], gfit$par[2], gfit$par[3]), col=2, lwd=3)
#lines(cg$x, sphercov(cg$x, sfit$par[1], sfit$par[2], sfit$par[3], sfit$par[4]), col=3, lwd=3)
legend("topright", legend=c("Exponential", "Gaussian", "Spherical"), col=1:3, lwd=2)

```

Task 2

- There are two other correlation functions built into the spatial package, the Gaussian covariance *gaucov* and the spherical covariance *sphercov*. Derive MLE for these two functions via numerical optimization. Turn in an AIC table for the models you fit and identify the best fitting model that you will use for Kriging

```
## define the distance bins we want to use for the fit
gaufit <- function(theta){ ## negative log likelihood function
  d = theta[1]
  alpha = theta[2]
  se = theta[3]
  sigma = theta[4]
  -sum(dnorm(cg$y[rng],gaucov(cg$x[rng],d,alpha,se),sigma,log=TRUE))
}
gfit <- optim(c(2,0.1,1,0.01),gaufit) ## numerical optimization
```

```
## Warning in dnorm(cg$y[rng], gaucov(cg$x[rng], d, alpha, se), sigma, log =
## TRUE): NaNs produced
```

```
gfit
```

```
## $par
## [1] 1.39858836 0.42649604 0.86171839 0.03273918
```

```
##
## $value
## [1] -20.00104
##
## $counts
## function gradient
##      203      NA
##
## $convergence
## [1] 0
##
## $message
## NULL
```

```
sphfit <- function(theta){    ## negative log likelihood function
  d = theta[1]
  alpha = theta[2]
  se = theta[3]
  sigma = theta[4]
  -sum(dnorm(cg$y[rng],sphercov(cg$x[rng],d,alpha,se),sigma,log=TRUE))
}
sffit <- optim(c(2,0.1,1,0.01),sphfit)    ## numerical optimization
```

```
## Warning in sqrt(1 - r^2): NaNs produced
```

```
## Warning in asin(r): NaNs produced
```

```
## Warning in sqrt(1 - r^2): NaNs produced
```

```
## Warning in asin(r): NaNs produced
```

```
## Warning in sqrt(1 - r^2): NaNs produced
```

```
## Warning in asin(r): NaNs produced
```

```
## Warning in sqrt(1 - r^2): NaNs produced
```

```
## Warning in asin(r): NaNs produced
```

```
## Warning in sqrt(1 - r^2): NaNs produced
```

```
## Warning in asin(r): NaNs produced
```

```
## Warning in sqrt(1 - r^2): NaNs produced
```

```
## Warning in asin(r): NaNs produced
```

```
## Warning in sqrt(1 - r^2): NaNs produced
```

```
## Warning in asin(r): NaNs produced
```


[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

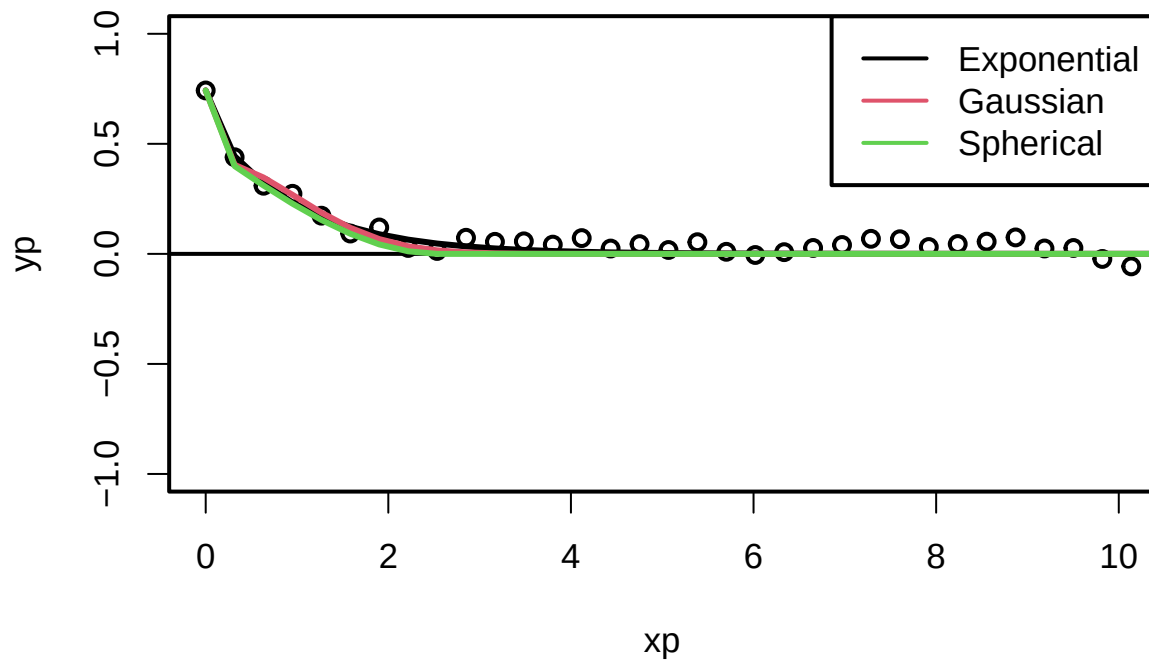
[illegible]

```
## Warning in sqrt(1 - r^2): NaNs produced
## Warning in asin(r): NaNs produced
## Warning in sqrt(1 - r^2): NaNs produced
## Warning in asin(r): NaNs produced
## Warning in sqrt(1 - r^2): NaNs produced
## Warning in asin(r): NaNs produced
## Warning in sqrt(1 - r^2): NaNs produced
## Warning in asin(r): NaNs produced
## Warning in sqrt(1 - r^2): NaNs produced
## Warning in asin(r): NaNs produced
## Warning in sqrt(1 - r^2): NaNs produced
## Warning in asin(r): NaNs produced
## Warning in sqrt(1 - r^2): NaNs produced
## Warning in asin(r): NaNs produced
## Warning in sqrt(1 - r^2): NaNs produced
## Warning in asin(r): NaNs produced
```

```
sfit
```

```
## $par
## [1] 2.5443433 0.3402727 0.8616668 0.0331835
##
## $value
## [1] -19.86668
##
## $counts
## function gradient
##      253      NA
##
## $convergence
## [1] 0
##
## $message
## NULL
```

```
par(lwd=2,cex=1.1)
cg <- correlogram(surf2,300,xlim=c(0,10))
lines(cg$x,expcov(cg$x,efit$par[1],efit$par[2],efit$par[3]),lwd=3)
lines(cg$x,gaucov(cg$x,gfit$par[1],gfit$par[2],gfit$par[3]),col=2,lwd=3)
lines(cg$x,sphercov(cg$x,sfit$par[1],sfit$par[2],sfit$par[3],sfit$par[4]),col=3,lwd=3)
legend("topright",legend=c("Exponential","Gaussian","Spherical"),col=1:3,lwd=2)
```



```
E.AIC <- 2*4 - 2*efit$value
S.AIC <- 2*4 - 2*sfit$value
G.AIC <- 2*4 - 2*gfit$value
```

```
E.AIC
```

```
## [1] 51.28906
```

```
S.AIC
```

```
## [1] 47.73335
```

```
G.AIC
```

```
## [1] 48.00208
```

3. Explain the figure generated from the previous box of code. What is being plotted (axes, data, model)? How would you interpret this figure? How would you interpret the parameters of the best-fit model? What is the range? Note: you do not need to include the Gaussian and spherical lines if you did not fit them in the previous step.

This figure shows the correlogram for all three models. All three models follow a similar pattern. This indicates that one covariance structure is not too much better than another. It looks like spherical might be

slightly better, but again, the difference is small. The range is 10, the number of distance bins we want to fit.

Now that we have a covariance function we are ready to proceed with Kriging. The code below is based on the `expcov` covariance function and a 0th order polynomial, but **you should use the polynomial you selected previously and the optimal covariance function.**

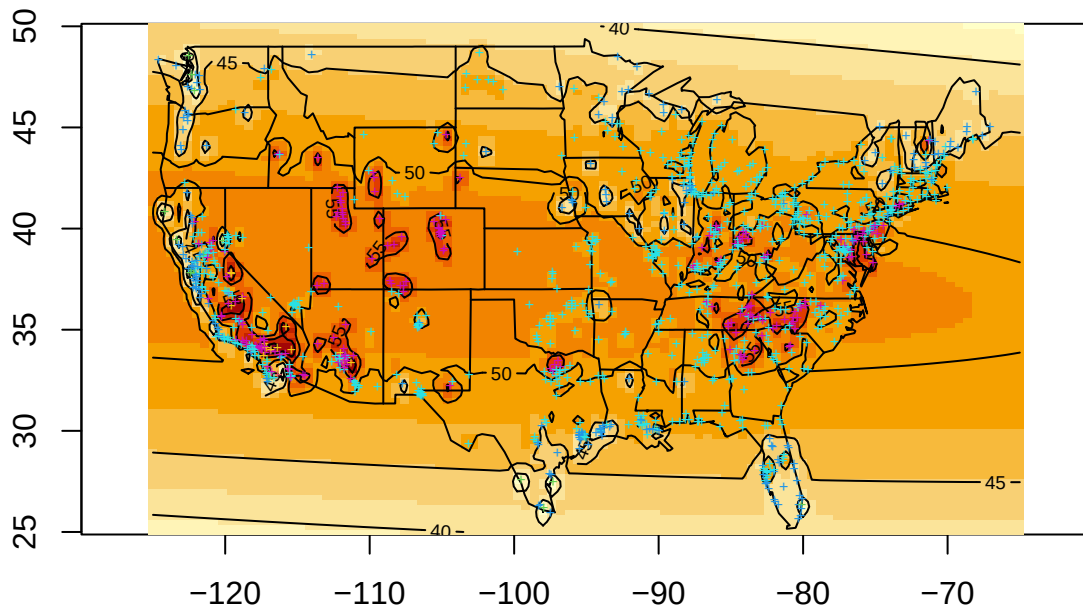
```
KrExp <- surf.gls(2,sphercov,dat$Lon,dat$Lat,dat$Mean,
                 d=efit$par[1],alpha=efit$par[2],se=efit$par[3]) ##construct surface

## Warning in sqrt(1 - r^2): NaNs produced

## Warning in asin(r): NaNs produced

pr3 <- prmat(KrExp,-125,-65,25,50,100) ## predict to a matrix
image(pr3,main="Krige",cex.main=2,asp=70/50) ## image of predictions
contour(pr3,add=TRUE,lwd=1) ## conv to ppb ## contours of prediction
map("state",add=TRUE)
points(dat$Lon,dat$Lat,pch="+",col=dat$Mean/10,cex=0.5)
```

Krige



```
se3 <- semat(KrExp,-125,-65,25,50,150) ## error matrix
```

The `surf.gls` function is similar to the `surf.ls` function but it differs in that it takes the spatial autocorrelation into account. The arguments are the same as for `surf.ls` with the addition that we specify the name of the

autocorrelation function in the second slot and the parameters to that function in the last few slots. The *prmat* function is also analogous to the *trmat* function but instead of just providing the trend, it provides a matrix that includes both the trend and the autocorrelation. In other words *prmat* is the function that actually does the kriging. The function *semt* is also very similar to *prmat* and *trmat* except that while *prmat* returns the mean of the prediction *semt* returns the standard error of the predictions.

Task 3

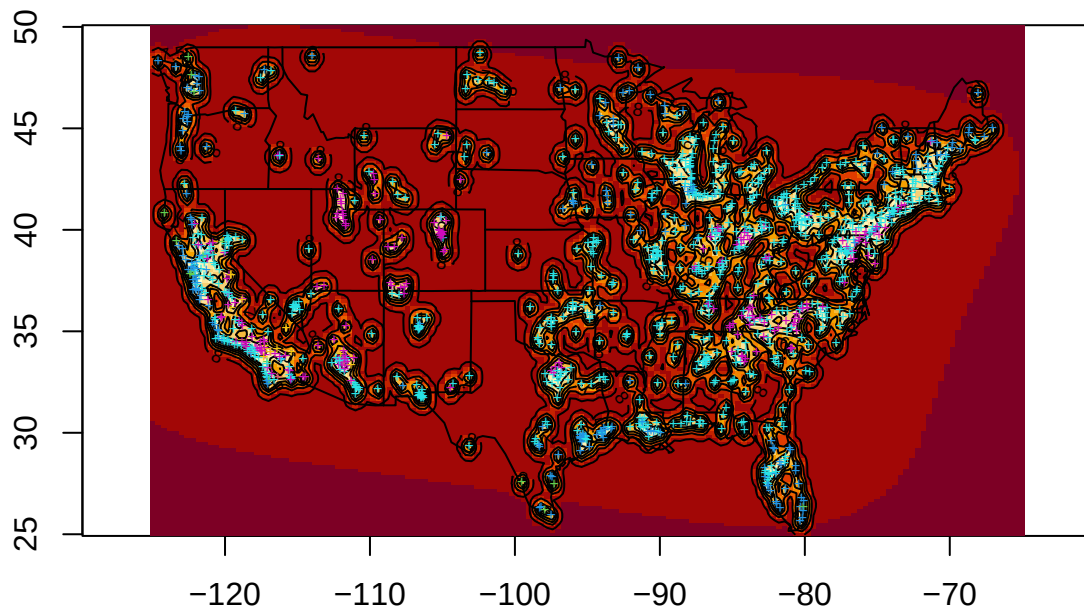
4. Include your kriged map of mean US ozone for 2008 in your answer. Where in the country has the highest ozone levels?

Map above. Highest ozone levels seem to be between 34 and 43 degrees latitude, and with a western bias. Locally high ozone levels are present around cities, especially in the rockies.

5. Generate a second map of mean US ozone but this time include contours based on the error matrix rather than the prediction matrix. Where are ozone levels most uncertain?

```
image(se3,main="Krige",cex.main=2,asp=70/50)           ## image of predictions
contour(se3,add=TRUE,lwd=1) ## conv to ppb             ## contours of prediction
map("state",add=TRUE)
points(dat$Lon,dat$Lat,pch="+",col=dat$Mean/10,cex=0.5)
```

Krige



There is a lot of uncertainty. Mostly in areas farther from places of measurement which makes sense.

6. There are a few reasons that one might be skeptical of this initial exploratory map. Name at least two and describe ways the model could be improved (without collecting additional data).

One could explore different interpolation surfaces. Also it may be worth trying higher order polynomials (>2) and see if it improves the model using the same steps done in this tutorial.